

The logo for 3SHAKE, featuring a stylized '3' with a small colorful icon (yellow, green, and blue) followed by the word 'SHAKE' in white capital letters.

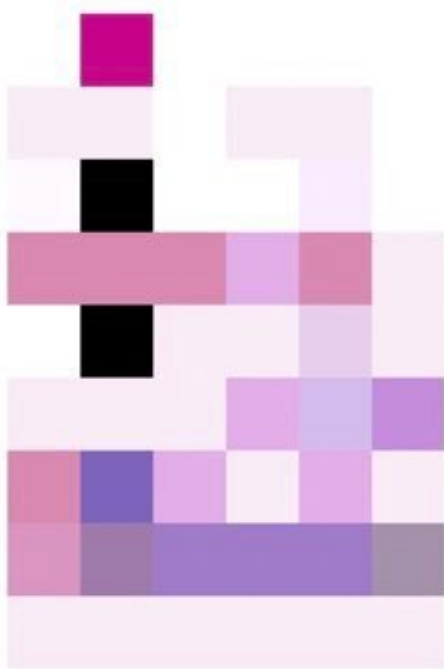
3SHAKE

30分でわかる Effective Platform Engineering

書籍の全体像を掴む

2026/XX/XX 3-shake SRE Tech Talk

@nwiizo 30min



nwiizo

株式会社スリーシェイクでプロのソフトウェアエンジニアをやっているものです。格闘技、読書、グラビアが趣味でよく本を紹介しています。

技術書翻訳を手がけるたび、わかることが1つ増えるのと引き換えに、わからないことが3つ増えていく。

インターネット上では **nwiizo** を名乗り、ブログ「**じゃあ、おうちで学べる**」を運営しています。X / GitHub もこのIDでやっています。

about 3-shake



Mission

インフラをシンプルにしてイノベーションが起こりやすい世界を作る

Vision

労苦（Toil）を無くすサービスを適正な価格で提供し続ける

SRE



スリーク

日本のSREをリードする

SRE総合支援からセキュリティ対策を全方位支援

Data Engineering



レコナー

**あらゆるサービスを
連携するハブになる**

クラウド型ETL/データパイプ
ラインサービスの決定版

Security



セキュリファイ

**事業者が抱える
セキュリティリスクを無くす**

ワンストップで実現する
セキュリティ対策

HR (Engineer Hiring)



リランス

**良いエンジニアに
良い案件を**

フリーランスエンジニアに
「今よりいい条件」を



Sreakeのお仕事

クラウドネイティブなアプローチで、お客様の事業をより安全に、競争力のあるサービスへ

提供サービス

SRE/DevOps支援

- Kubernetes構築・運用
- クラウドネイティブ化推進
- Observability導入

プラットフォームエンジニアリング

- 内部開発者プラットフォーム
- 開発者体験の改善
- セルフサービス基盤構築

データ活用支援

- データ基盤構築
- BigQuery/Snowflake
- 分析基盤最適化

ご依頼・ご相談お待ちしております

<https://sreake.com/>



この発表で解決できること

こんな悩みを持っていませんか？

「プラットフォームを作ったが使われない」

→ 開発者を顧客として扱うプロダクトデリバリーモデルを紹介します。「作ったから使え」ではなく「使いたくなるものを作る」発想への転換

「DevOpsとどう違うのか説明できない」

→ プラットフォームエンジニアリングはツール導入ではなく、ソフトウェア・デファインドな内部プロダクト開発であることを整理します

「成功をどう測ればいいのか分からない」

→ 認知負荷・本番環境への道・採用率といった測定可能な指標で、プラットフォームの価値を経営層に語る武器を提供します

今日の30分で「この本、読んでみよう」と思えるようになること

※30分で10章を扱う以上、各手法の実践詳細は書籍本体に委ねます。原則は組織規模を問わず適用できます



書籍の概要

Effective Platform Engineering

- 著者: Ajay Chankramath, Sean Alvarez, Bryan Oliver, Nic Cheneweth
- 翻訳: 株式会社スリーシェイク
- 原書: Manning Publications, 2024

本書の一言要約

プラットフォームエンジニアリングを「ツールの寄せ集め」ではなく「内部開発者向けのプロダクト開発」として捉え直すための実践書。プロダクト思考・ソフトウェア・デファインド・認知負荷の3つを軸に、構築からスケーリングまでを通貫して扱う。



本日の流れ

1. **Part 1: 基礎** — なぜプラットフォームエンジニアリングか

2. **Part 2: 構築** — どう作るか

3. **Part 3: スケーリング** — どう育てるか

4. **まとめ** — 何を持ち帰るか

書籍の構成に沿って、各パートの核心を紹介します



書籍は3つのパート・10章で構成されている

Part	テーマ	章	問い	主要な概念・手法
1. 基礎	Why	Ch.1-3	なぜ・何を・どう測るか	プロダクトデリバリーモデル、SDP、認知負荷
2. 構築	How	Ch.4-8	何を作り、どう統治・観測・拡張するか	ガバナンス、オブザーバビリティ、コントロールプレーン
3. スケーリング	Scale	Ch.9-10	どう育てるか	イベント駆動、プロダクト進化、文化変革

基礎 → 構築 → スケーリング の流れ。「内部プロダクトを進化させる」が一貫したメッセージ



Part 1: 基礎

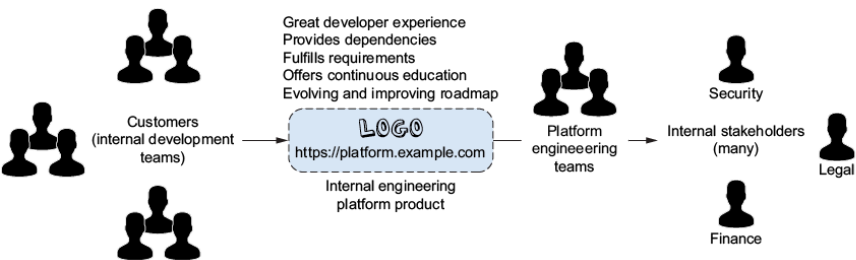
Ch.1-3

最初の問い：「プラットフォームエンジニアリングとは何か？」

Ch.1 プラットフォームエンジニアリングの定義

定義：開発者にインフラ・セキュリティ・デプロイ機能へのセルフサービスアクセスを提供する内部ソフトウェアシステムを構築する実践

「ツールを集めて開発者に渡す」ではない。「開発者を顧客とした内部プロダクトを開発すること。プロダクトマネジメント・アーキテクチャ・エンジニアリングを統合した専任チームが、完全なオーナーシップを持って作り続ける。



DevOps/SRE/開発者体験との関係

領域	焦点	プラットフォームとの関係
DevOps	開発と運用の壁を壊す文化	文化基盤として前提
SRE	信頼性をエンジニアリングで担保	プラットフォーム上で実装
開発者体験(DX)	開発者の摩擦を減らす	プラットフォームの成果指標の1つ

Figure 1.2 統一されたプロダクトチームとして働くプラットフォームエンジニア より引用

DevOpsの「次」ではなく、DevOpsを成立させるための内部プロダクトという位置づけ



Epetechの現場――どこかで見た景色

本書はEpetech社（架空のヘルスケアテック・100名超のエンジニア組織）を事例に展開する。「どこかで見たことがある」と感じる読者は多いはず。

Epetechで起きていたこと

観察された事象	数値・規模
開発者の業務時間の使い道	半分が調整作業（DNS、FW、ストレージ、監視等）
インフラ変更のハンドオフ数	最大4チームを経由
デプロイプロセスの所有権	別チームが管理。リリースに数週間
結果	本番インシデント増加 → 顧客満足度低下

「最適化は各チームの責任範囲のみ許可」――これが組織全体の摩擦を増やしていた



DevOpsだけでは足りない理由

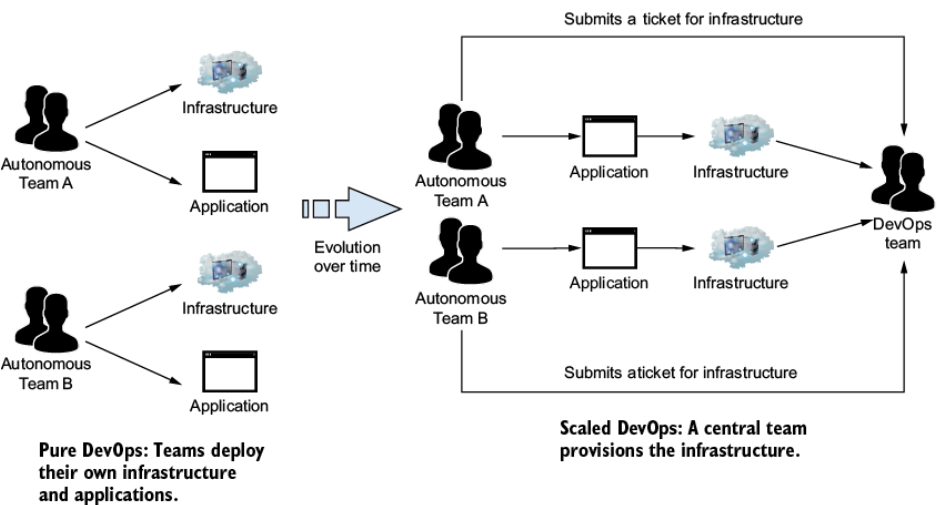


Figure 1.1 DevOps文化を採用する企業における典型的な出発点 より引用

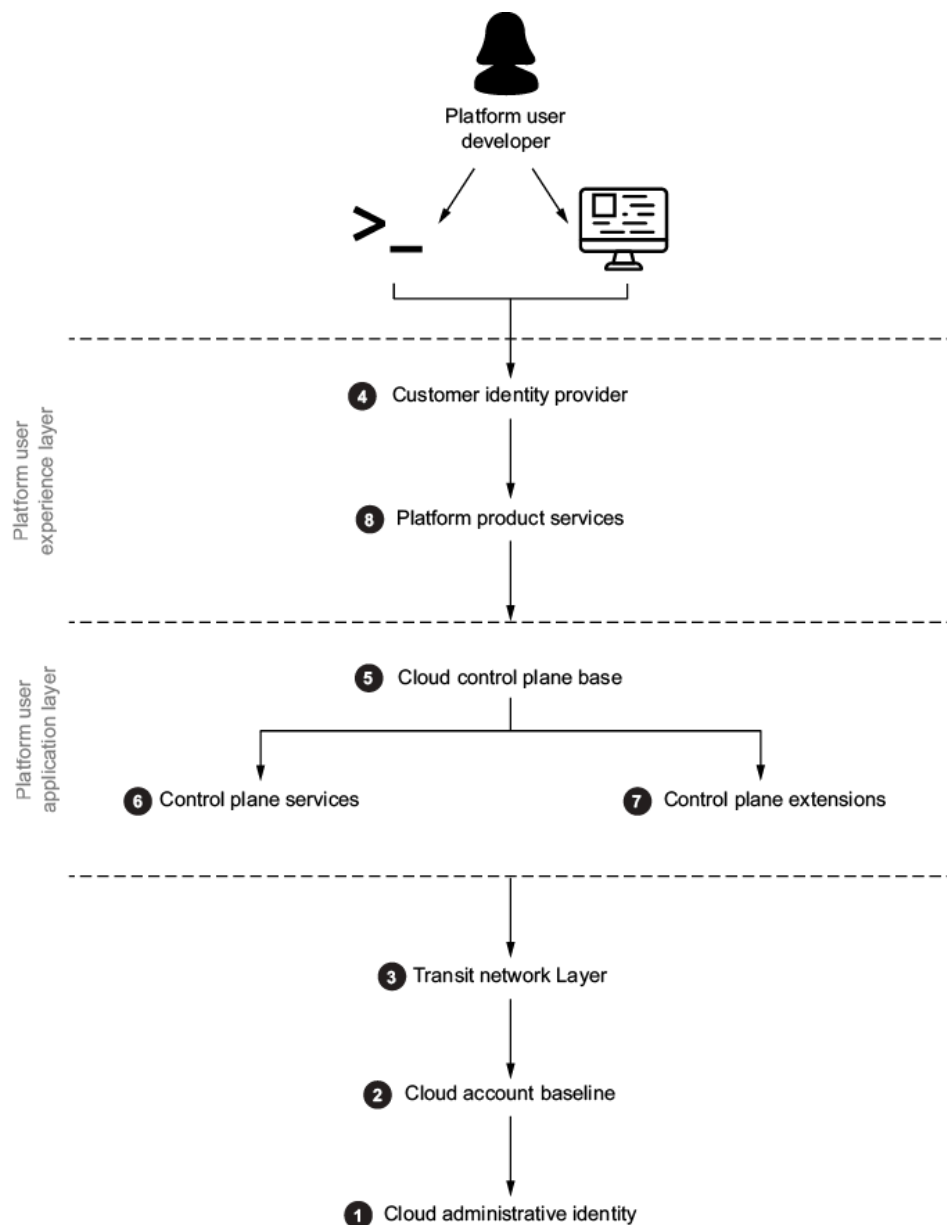
多くの組織のDevOps導入はツールに偏り、プロセスとエンジニアリング文化の変革は十分ではなかった。「You build it, you run it」のスローガンの下、各開発チームが個別にインフラを抱え込み、車輪の再発明と認知負荷の爆発を招いた。

よくある現状

状態	何が起きるか
各チームが個別にCI/CDを構築	同じパイプラインが乱立、知見が共有されない
インフラ変更が4チームを経由	1リリースに数週間。責任は自エリアのみ
開発者が10種以上のツールを使い分け	認知負荷が上限を超え、ドメインに集中できない

DevOpsの理想は正しい。ただし「全員にフルスタックを求める」と認知負荷で潰れる





プラットフォームは1つの巨大なシステムではなく、独立した8つのプロダクトドメインの集合体。構築の順序にも依存関係がある。基盤から積み上げないと崩れる。

#	ドメイン	役割
1	クラウドアカウント基盤	アカウント・課金・最小権限の土台
2	ネットワーク	トランジット、セグメンテーション
3	アイデンティティ	顧客ID・サービスアカウント
4	コントロールプレーン	プロビジョニングと統治の中核
5	オブザーバビリティ	メトリクス・ログ・トレース
6	アプリケーション配信	ビルド・デプロイ・リリース
7	データ・セキュリティ	データ保護・シークレット管理
8	開発者ポータル	セルフサービスのフロントドア

Ch.2 ソフトウェア・デファインドのプロダクトとアーキテクチャ

プラットフォームエンジニアリングの核心は、ソフトウェアエンジニアリングの一カテゴリであるという認識。インフラを「クリックして設定するもの」から「コードで定義し、テストし、リリースするプロダクト」へ。

よくあるアンチパターン

アンチパターン	何が起きるか	正しいアプローチ
Ops出身者が手動運用の延長で構築	スクリプトの寄せ集め、再現性なし	プラットフォームをソフトウェアプロダクトとして開発
ツール選定が単発プロジェクト化	数年使い回されるレガシー化	進化を前提とした継続的な投資
インフラ変更を直接ユーザーに渡す	ハンドオフ地獄、責任の押し付け合い	プロダクトオーナーが体験を所有

「インフラの調達」ではなく「プロダクトの開発」。発想の転換が出発点



プロダクトデリバリーモデル

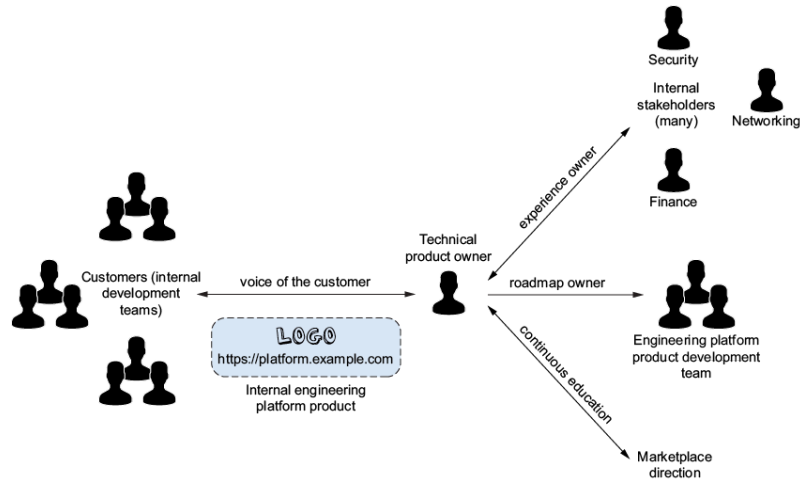


Figure 2.1 テクニカルプロダクトオーナーは顧客の声として機能する より引用

開発者を「内部の顧客」として扱い、テクニカルプロダクトオーナー(TPO)が顧客の声として機能する。提供される価値を継続的に評価し、必要なものだけを作る。

TPOの役割

顧客の代弁

開発者が何に困っているかを継続的にヒアリング。仮説ではなく観察に基づいて優先順位を決める。

価値の計測

機能をリリースして終わりではなく、採用率・満足度・効果を計測。使われない機能は削る勇気を持つ。

ロードマップ

MVPから始めて段階的に進化。「全部入り」を最初から目指さない。



使われないプラットフォームは存在価値がない。採用は強制ではなく選ばれることで成立する

進化的プラットフォームアーキテクチャ

「最初から完璧なアーキテクチャ」は存在しない。プラットフォームはユーザー数・要件・スケールに応じて進化させる。設計時の判断は「今のための最善」であり、半年後には書き換える前提で作る。

進化のための設計原則

原則	意味
抽象化	コントロールプレーンの実装をユーザーから隠す
イベント駆動	同期的な依存を減らし、コンポーネント単位で進化可能にする
境界の明確化	プロダクトドメイン間の契約を定義し、内部実装を入れ替え可能
テスト可能	全変更がコードと同じくCI/CDを通る

「未来の自分が困らないように」設計する。今のためだけに作るとすぐに負債化する



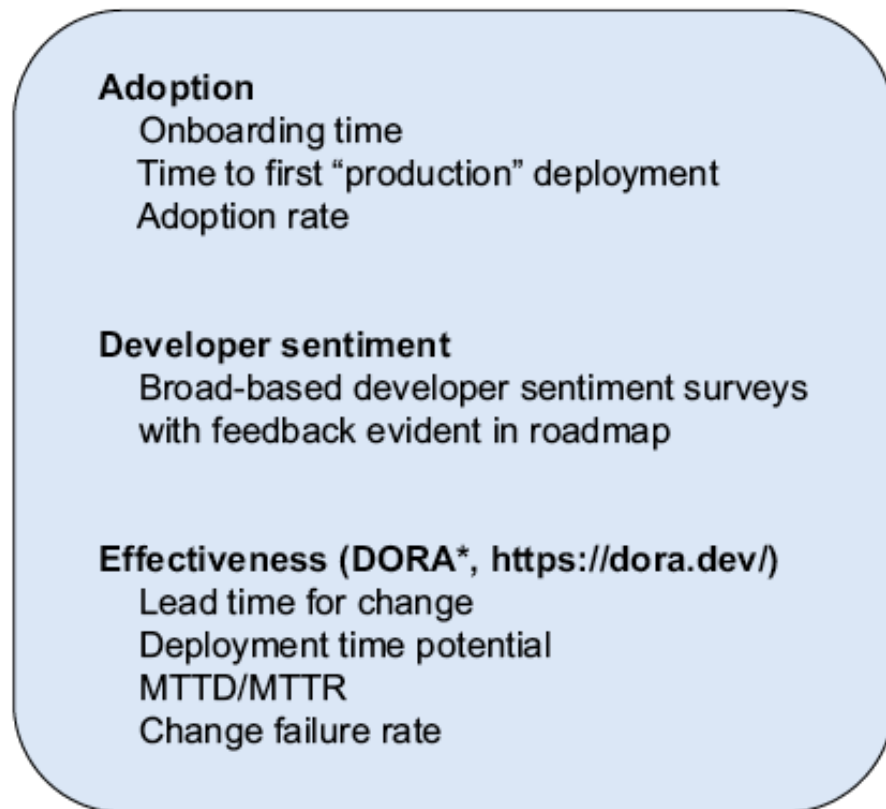


Figure 3.6 あらゆるプラットフォームの共通計測項目 より引用

「採用されないプラットフォーム」は失敗する。経営層が予算を出し続けるためには、価値を数値で語れなければならない。プラットフォームの計測には3つの軸がある——開発者の生産性、認知負荷、組織の健全性。収益だけでは語れない。

3つの計測軸

開発者生産性

リードタイム、デプロイ頻度、変更失敗率、MTTR（DORA 4キーマトリクス）

認知負荷

開発者が把握すべきツール・概念の数。ドメインに集中できる時間の割合。

組織の健全性

エンジニアの満足度、採用率、離職率、ステークホルダー整合性



認知負荷を計測すること

認知負荷 = 任意の時点でワーキングメモリが使う精神的努力の総量。Geraldine Weinberg 『The Psychology of Computer Programming』 (1971) 以来、ソフトウェア開発の本質的な制約として議論されてきた。

プラットフォームが減らすべき負荷

負荷の種類	例	プラットフォームの対応
外在性（不要）	ツールの設定方法を覚える、社内固有の運用手順	セルフサービス化、ドキュメント自動生成
関連性（必要）	ドメイン知識、ビジネス要件	開発者のここに集中できるようにする
本質的（やむを得ない）	分散システムの複雑性、トレードオフ判断	適切な抽象化で隠蔽、ただしブラックボックス化はしない

外在性負荷を減らせば、関連性負荷に振れる脳の容量が増える。これがプラットフォームの本質的価値



本番環境への道を地図にする

Path to Production = アイデアから本番稼働までの全ステップを可視化したバリューストリームマップ。コンウェイの法則がチーム境界を決め、その境界がそのままバリューストリームの摩擦になる。

計測の対象

指標	何を見るか
リードタイム	コミットから本番までの時間
デプロイ頻度	安全に頻繁にデプロイできるか
変更失敗率	デプロイの何%が問題を起こすか
MTTR	障害から復旧までの平均時間
待ち時間 vs 実作業時間	バリューストリームのどこで時間が溶けているか

「忙しい」は仕事をしている時間ではなく、待っている時間が長い証拠であることが多い



基礎はわかった。では何をどう作るか

Part 1でプラットフォームをプロダクトとして扱い、ソフトウェア・デファインドで作ри、3軸で計測するという土台を見てきました。問いはここから具体になる。

ガバナンスはどう実装する？オブザーバビリティはどこまで作り込む？コントロールプレーンの中身は？Part 2では、**実際に何を作るか**ー統治・観測・配信・コントロールプレーンを順に扱います。

Part 2: 構築

Ch.4-8

次の問い：「実際に何を、どの順番で作るか？」

Ch.4 ガバナンス、コンプライアンス、信頼

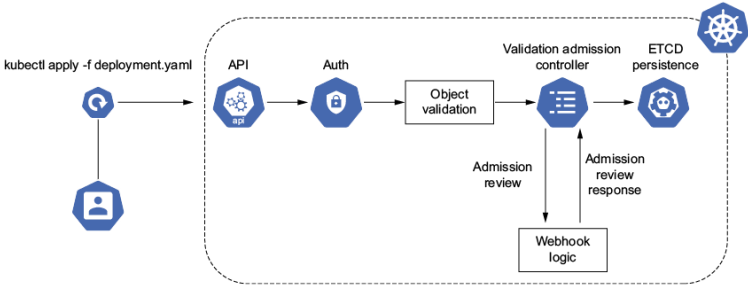


Figure 4.3 Kubernetes Admission Controller より引用

セルフサービスを開放するほど、ガバナンスをコードで実装する必要が出てくる。レビュー会議で守るのではなく、ポリシーで自動的に守る。これがプラットフォームの信頼を支える。

ガバナンスの3層

層	何をするか	実装例
ポリシーアズコード	コンプライアンス要件をコードで宣言・検証	OPA/Gatekeeper、Kubernetes Admission Controller
プロベナンス	コードからデプロイまでの来歴を自動記録	SLSA、署名されたアーティファクト
信頼の管理	プラットフォーム自身が「誰が何をできるか」を管理	IDフェデレーション、最小権限

「ガバナンス vs 自律性」はトレードオフではない。コード化すれば両立できる



開発者の自律性とガードレールの両立

自律性の敵は「ゲートキーパー」。セキュリティチームやアーキテクトが各リリースを手動で承認する仕組みは、開発者の自律性を奪い、ボトルネックになる。代わりに、Admission Controllerがデプロイメント時に自動で検証する仕組みを作る。

Before / After

状態	Before（ゲート型）	After（ガードレール型）
検証タイミング	リリース直前のレビュー会議	コミット時、PR時、デプロイ時に自動検証
失敗時の影響	数日～数週間の遅延	数秒で開発者にフィードバック
検証の一貫性	レビュアーの気分次第	全環境・全チームで同じルール
ガバナンスの可視性	暗黙知	コードとしてバージョン管理

ゲートを置くな、ガードレールを敷け。守りながらスピードを落とさない設計



ポリシーアズコードの実装――OPAとRego

「テストカバレッジ90%以上を全パイプラインで強制したい」――これを集中管理されたDevOpsチームではなく、各チームが自律しながら満たすにはポリシーエンジンが必要。

主要な選択肢

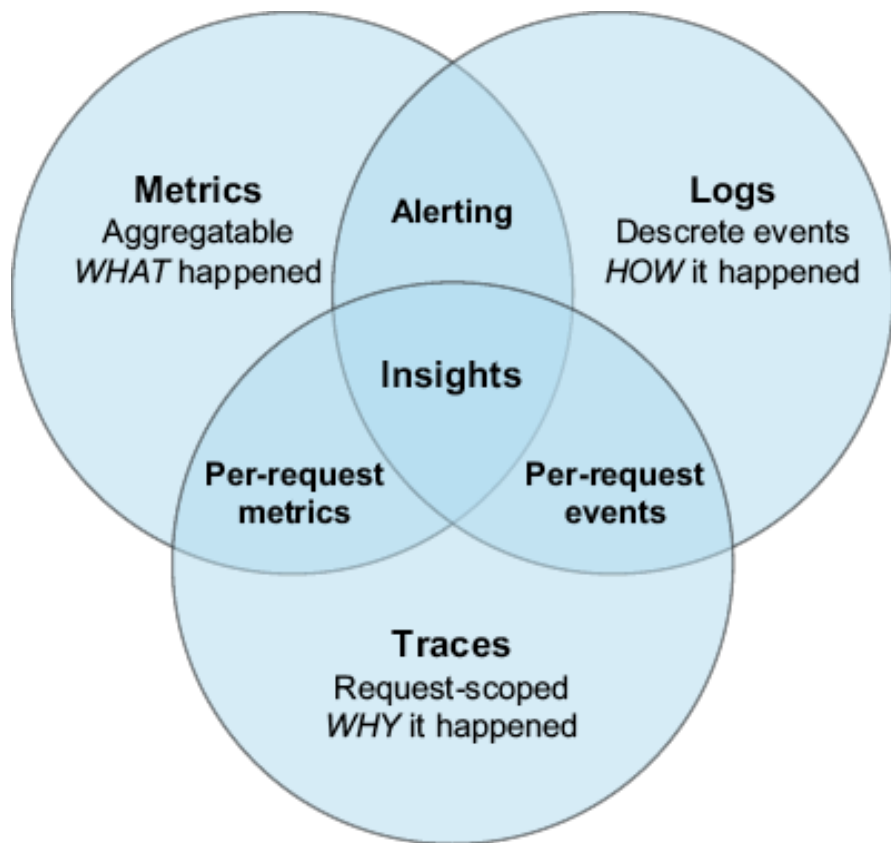
ツール	特徴	適用範囲
Open Policy Agent (OPA) + Rego	CNCF卒業プロジェクト、宣言的ポリシー言語Rego	Kubernetes, Terraform, API認可など全般
Gatekeeper	OPAをKubernetesアドミッションコントローラに特化	Kubernetesリソース検証
Kyverno	YAMLでポリシー記述、Rego不要	Kubernetes中心
HashiCorp Sentinel	Terraform環境向けのベンダーソリューション	IaC全般

事前 vs 事後

事前チェック（アドミッション時）で違反を弾くか、事後スキャンで違反を検出してアラートする。パイプラインの所有権を開発者から奪わずにガバナンスを成立させるのがポリシーアズコードの本質。

共通標準（Regoなど）を選ぶ理由は、Kubernetes以外の領域にもポリシーを横展開できるため





オブザーバビリティはメトリクスを超え、ログとトレースを含む。「何が起きているか」「なぜ起きたか」「どうやって今の状態になったか」を答えるためのデータ基盤。プラットフォームの初期では同梱で十分だが、スケールするにつれ独立したオブザーバビリティ基盤が必要になる。

Figure 5.1 オブザーバビリティはメトリクス・ログ・トレースを含む より引用

3つのテレメトリ

種類	答える問い	主な用途
メトリクス	何が、いつ、どれくらい起きたか	アラート、ダッシュボード、SLO監視
ログ	詳細に何が起きたか	デバッグ、監査、セキュリティインシデント
トレース	リクエストがどう流れたか	レイテンシ分析、依存関係の理解



SLI/SLO/SLAとプラットフォームの責任

プラットフォームは内部プロダクトであり、利用者（開発チーム）に対してソフトウェアとしての契約を持つ。「ベストエフォート」では信頼されない。

階層関係

SLI

実際に計測する指標。可用性、レイテンシ、エラー率。

SLO

社内に約束する目標。「99.9%」「p99 200ms以下」。

SLA

外部に契約する保証。違反時のペナルティを伴う。

よくある落とし穴

オブザーバビリティ基盤自体の障害監視を、その基盤の中で行ってしまう。オブザーバビリティプラットフォームは外部から観察する必要があるーでないと完全障害時にアラートが鳴らない。

SLOダッシュボードを公開することで「プラットフォーム側の問題ではない」を即座に示せる



Ch.6 エンジニアリングプラットフォームの構築

真のセルフサービス体験を提供する内部プロダクトを作る。アプリケーションの構想・設計・構築・リリース・運用を、開発者が摩擦なく行える状態を目指す。

ツール選定の評価軸

軸	問い
統合性	他のプラットフォーム要素と疎結合に組み合わせられるか
API/拡張性	コードから操作・自動化できるか
マルチテナンシー	チーム単位で隔離できるか
運用負荷	プラットフォームチーム自身が運用できるか
顧客と同じツールを使えるか	ドッグフーディングができるか

「ベストツール」ではなく「組み合わせやすいツール」を選ぶ。プラットフォームは進化するから



シークレット管理をセルフサービスにする実装

「チケットでシークレットをもらう」体験を完全に消す。Epetechの実装は自動オンボーディング+ローテーション+真の単一ソースの3点で構成される。

構成要素

コンポーネント	役割
Vault等のシークレットマネージャ	単一かつセキュアな権威あるソース。すべての自動化はここから取得
チームスペース自動払い出し	チームオンボーディング時にRBAC込みで作成。手動申請なし
OIDCトラスト/サービスアカウント	パイプライン用クレデンシャルを動的に発行・ローテーション
external-secrets-operator	Kubernetes Namespaceにシークレットを自動同期
Argo CD / Flux	GitOpsデプロイで上記を組み合わせる

設計原則

単一の権威あるソースでしかシークレットを取得しない。ローテーションを自動化することでセキュアな構成プラクティスの維持コストを劇的に下げる。「使う場所」を増やしてはいけない。「作って配る場所」が分散すると整合性が崩れる。

セルフサービス化はガバナンスの放棄ではない。むしろ自動化されたガードレールで強化される



パイプラインを「共有コードライブラリ」で標準化する

各チームが個別のCI/CDを持つと、ベストプラクティスの伝搬が止まる。解決策はパイプラインサーバー側のプラグインではなく、パイプラインコード側の共有ライブラリ。

ツール別の共有メカニズム

ツール	共有の単位
GitHub Actions	Reusable Workflows / Composite Actions
CircleCI	Orbs
GitLab CI	<code>include:</code> で外部ファイル参照
Tekton	Catalog tasks

なぜサーバー側プラグインを避けるか

サーバー（Runner）拡張型は脆弱でメンテナンスコストが高い。バージョン管理されたコードとして共有ライブラリを管理すれば、パイプラインコード自体がインフラ・アプリと同じリポジトリで進化できる。トリガーは「ソースコード変更」のみ。リポジトリ間でパイプラインがパイプラインを呼ぶのは不健全な結合の兆候。

共有ライブラリの社内開発も、統合テスト・バージョニング・リリースなどソフトウェア開発の規律で運用する



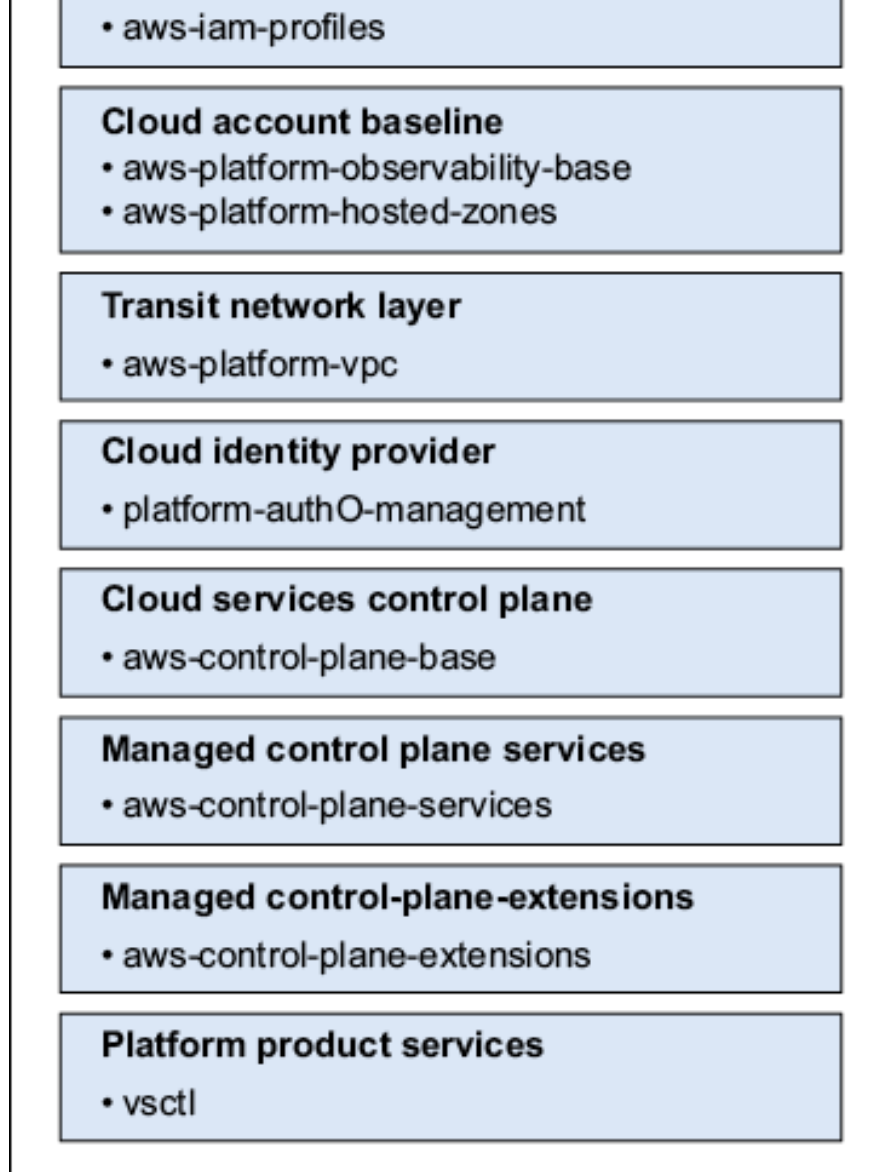


Figure 7.1 プラットフォームコントロールプレーンの基礎 より引用

コントロールプレーンが扱う4層

層	役割
---	----

Ch.8 コントロールプレーンのサービスとエクステンション

コントロールプレーンは1つの巨大システムではなく、サービスとエクステンションの集合体。 イベント駆動アーキテクチャを採用し、各統合・構成リソースがイベントに応答する形にすると、担当チームの移行とスケーリングが容易になる。

統合オーケストレーター

主要な顧客ペルソナ（チーム）を中心にモデル化された**単一のプライマリ統合オーケストレーター**が、標準イベントストリームを維持する。これがある限り、統合活動はプラットフォームとともに進化できる。

設計判断	選択肢A（避ける）	選択肢B（推奨）
サービス間連携	同期API呼び出し	イベント駆動、非同期メッセージング
状態管理	中央集中型の状態ストア	各サービスが自分の状態を持つ
拡張の方法	コア改修	エクステンションポイントで追加

コントロールプレーン自体に進化的アーキテクチャを適用する。これがスケーリングの前提



構築できた。次はスケーリング

Part 2ではガバナンス・オブザーバビリティ・コントロールプレーンを作りました。最初の数チーム、最初のいくつかのワークロードでは、これで動く。

しかし、組織が10チーム、100チーム、1000サービスへとスケールすると、最初のアーキテクチャは破綻し始める。同期通信は詰まり、ロールが機能境界を超え、文化が追いつかなくなる。Part 3では、**スケールに耐える進化**——アーキテクチャの組み換え、組織変革、プロダクトとしての進化を扱います。



Part 3: スケーリング

Ch.9-10

最後の問い：「どう育て続けるか？」

Ch.9 スケールを支えるアーキテクチャの変更

スケールするとアーキテクチャは破綻する。同期APIは詰まり、ロールは機能境界を超え、変更速度は鈍る。スケールフェーズではアーキテクチャを「組み直す」必要がある。

3つのスケーリング戦略

ロールのスケーリング

1人/1チームでは捌けなくなった責務を分割。テクニカルプロダクトオーナー複数化、ドメイン別チーム化。

オーケストレーションのスケーリング

中央集中型のオーケストレーターから、ドメイン別のエクステンション群へ。コア + プラグインの構造。

イベントストリーミング

同期呼び出しを非同期イベントに置き換え、コンポーネント間の結合度を下げる。

スケールの本質は「結合を減らすこと」。リソースを増やしても、結合があれば詰まる



イベントストリーミングが変える設計判断

プラットフォームの内部通信を「同期API」から「イベントストリーム」へ寄せると、各エクステンションが独立してスケール・進化できるようになる。

同期 vs イベント駆動

観点	同期APIの場合	イベント駆動の場合
障害の伝播	1サービスのダウンが連鎖する	バッファリングされ、上流に伝播しにくい
バージョンアップの調整	全サービスを揃えてリリース必要	各サービスが独自のペースで進化可能
監査・リプレイ	困難（ログの寄せ集め）	イベントログが事実上の単一の真実
デバッグ難易度	同期トレースは追やすい	非同期トレースの工夫が必要

イベント駆動は「正解」ではなく「トレードオフ」。スケールが必要な箇所に絞って適用する



Ch.10 プラットフォームプロダクトの進化

プラットフォームに「完成」はない。組織のニーズ、テクノロジーの進化、規制の変化に合わせて作り変え続ける。プロダクトとしての成功は、収益ではなく組織への影響で測る。

成熟度に応じた焦点の変化

フェーズ	焦点	主要メトリクス
立ち上げ	コアユースケースの自動化、初期採用者の獲得	採用チーム数、初回利用までの時間
拡大	カバー範囲の拡張、認知負荷の削減	採用率、開発者満足度、リードタイム短縮
成熟	効率化、コスト最適化、差別化要因への進化	TCO、ビジネス成果、人材の競争優位

「プロダクトとしてのプラットフォーム」は競争優位の源泉になる



Epetech変革の数字

変革の効果は技術指標だけでは語れない。Epetechの事例で本書は具体的な数字を示している。

Before / After の指標

指標	Before	After
コミット → デプロイの時間	数週間	数時間
本番環境の欠陥	基準	60%削減
開発者満足度	基準	40%向上
インフラ変更のハンドオフ	最大4チーム経由	チーム内で完結

組織側の変化（Team Topologiesの適用）

Stream-Aligned Team（eコマース、モバイル、在庫など）が価値ストリームを所有 / **Platform Team**が共通基盤を内部プロダクトとして提供 / **Enabling Team**がクラウド移行・新技術導入を伴走支援。

「数字で語る」ことが経営層からの継続投資を引き出す。これが計測の3軸を真剣に作る理由



文化的変革——従来のオペレーション世界からの脱却

技術だけ変えても、文化が変わらなければプラットフォームは形骸化する。旧来のOps文化（手動・チケット・ガードマン）を引きずったまま「プラットフォーム」と看板を書き換えても、開発者から見れば中身は同じ。

変えるべきマインドセット

Before	After
インフラチームが「守る」	プラットフォームチームが「使われる」
チケットでリクエストを受ける	APIで自律的にリクエストされる
安定性のために変更を遅らせる	安定性のために変更を頻繁にする
「うちのインフラは特殊」	「内部プロダクトとして開発する」

プラットフォームエンジニアリングは技術プロジェクトではなく、組織の働き方を変える取り組み

書籍全体のメッセージ

10章を通じて書籍が一貫して伝えているのは、プラットフォームエンジニアリングは「ツールの寄せ集めを置く」ではなく「内部開発者向けのプロダクトを開発し続ける」という認識の転換。

3つの統合

軸	内容
プロダクト	開発者を顧客として扱い、価値を計測する
ソフトウェア	インフラをコードで定義し、テスト・進化させる
組織	チーム構造・文化・スキルを揃える

プラットフォームを作るな。プラットフォームを「プロダクト」として育てよ。



どこから読むか

本音を言えば、Ch.1からCh.10まで順番に読んでほしい。著者も翻訳者も、その順番で読んだときに最も学びが深くなるよう設計している。

ただ、時間がない場合は目的別に以下のルートから入ってください。

これから始める

Ch.1 → Ch.2 → Ch.6

定義 → ソフトウェア・デファイン
ド → 構築の起点

すでに着手中

Ch.3 → Ch.4 → Ch.5

成功の計測 → ガバナンス → オブザ
ーバビリティ

規模で詰まっている

Ch.7 → Ch.9 → Ch.10

コントロールプレーン → スケーリン
グ → 進化



本日のまとめ

書籍の全体像: 基礎 → 構築 → スケーリングの3パート・10章。プラットフォームエンジニアリングを「内部開発者向けプロダクト」として捉え直し、プロダクト思考・ソフトウェア・デファインド・認知負荷の3軸で組み立てる。

翻訳者として一番伝えたいこと

プラットフォームエンジニアリングの本質は、「便利な内部ツールを置くこと」ではない。**開発者が認知負荷から解放されてドメインに集中できる状態を、プロダクトとして作り、進化させ続けること。**完成形を目指す瞬間にプラットフォームは陳腐化する。問い続けるべきは「使われているか」「価値が出ているか」「組織は変わったか」だ。

「全部読む時間がない」への回答

Ch.1（定義）、Ch.3（計測）、Ch.6（構築）——この3章だけで実践に着手できる。Ch.1だけだと「概念は分かったが何を作るかわからない」、Ch.6だけだと「作るものは分かったが価値を語れない」。**Ch.1 → Ch.3 → Ch.6** の順で読むのが最短経路。

プラットフォームは作るものではなく、育て続けるもの。今日から始めれば、組織はもう変わり始めている。



参考資料

- [Effective Platform Engineering](#) - Ajay Chankramath, Sean Alvarez, Bryan Oliver, Nic Cheneweth (Manning, 2024)
- [Effective Platform Engineering 日本語版](#) - 株式会社スリーシェイク訳
- [Team Topologies](#) - Matthew Skelton, Manuel Pais (IT Revolution, 2019)
- [Platform Engineering](#) - コミュニティとリソース
- [The Psychology of Computer Programming](#) - Gerald M. Weinberg (1971)
- [Accelerate](#) - Nicole Forsgren, Jez Humble, Gene Kim (IT Revolution, 2018)
- [Building Microservices, 2nd Edition](#) - Sam Newman (O'Reilly, 2021)
- [Wardley Maps](#) - Simon Wardley
- [The DevOps Handbook](#) - Gene Kim et al. (IT Revolution, 2021)
- [Internal Developer Platform](#) - IDPの定義とリファレンス



The logo for 3 SHAKE, featuring the number '3' followed by a small icon of three overlapping colored triangles (yellow, green, blue) and the word 'SHAKE' in a bold, white, sans-serif font.

3 SHAKE

ありがとうございました

ご質問・ご相談はお気軽にお問い合わせください

@nwiizo | <https://3-shake.com>